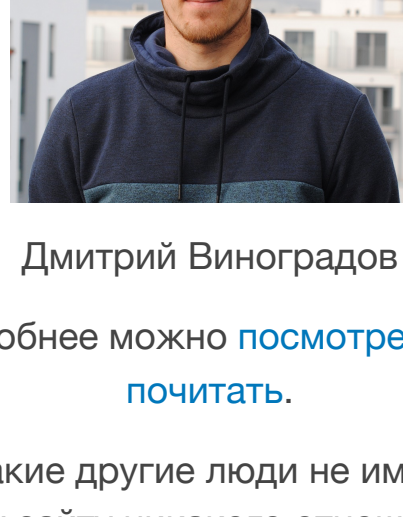


Language



Автор сайта



Дмитрий Виноградов

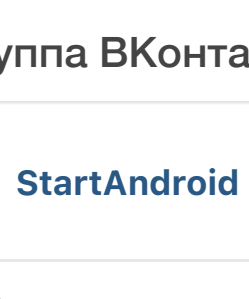
Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50% на набор курсов:

Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

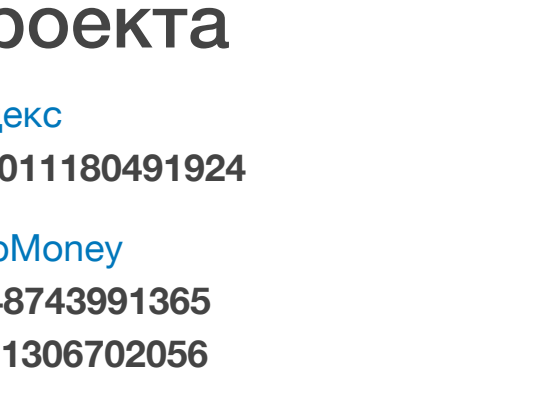
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

[Яндекс](#)
410011180491924

[WebMoney](#)
R248743991365
Z551306702096

[Paypal](#)

Урок 3. Modifier

14 декабря 2022

В этом уроке разберемся, где искать нужный нам атрибут элемента - в параметрах функции или в Modifier.

Modifier

На прошлом уроке мы использовали параметры функции Text, чтобы задать размер и цвет текста. Также мы выяснили, что у Text нет явных параметров, чтобы задать размер самого элемента или фоновый цвет. Для этого необходимо использовать параметр Modifier.

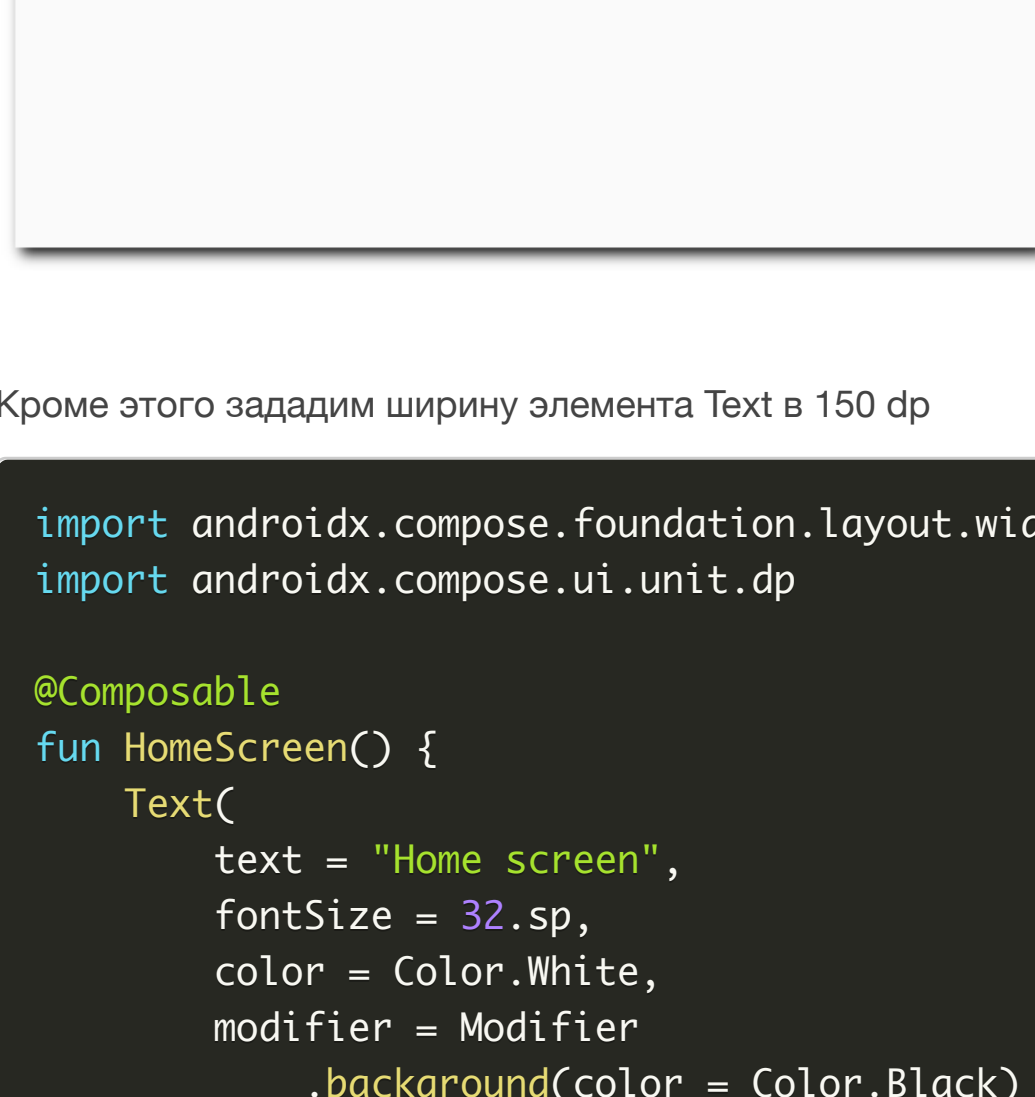
Давайте добавим черный фоновый цвет для нашего текста. А сам текст сделаем белым.

```
import androidx.compose.runtime.Composable
import androidx.compose.material.Text
import androidx.compose.ui.unit.sp
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.Modifier
import androidx.compose.foundation.background

@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 32.sp,
        color = Color.White,
        modifier = Modifier.background(color = Color.Black)
    )
}
```

В параметр modifier передаем Modifier.background с черным цветом.

Запускаем приложение. Результат:



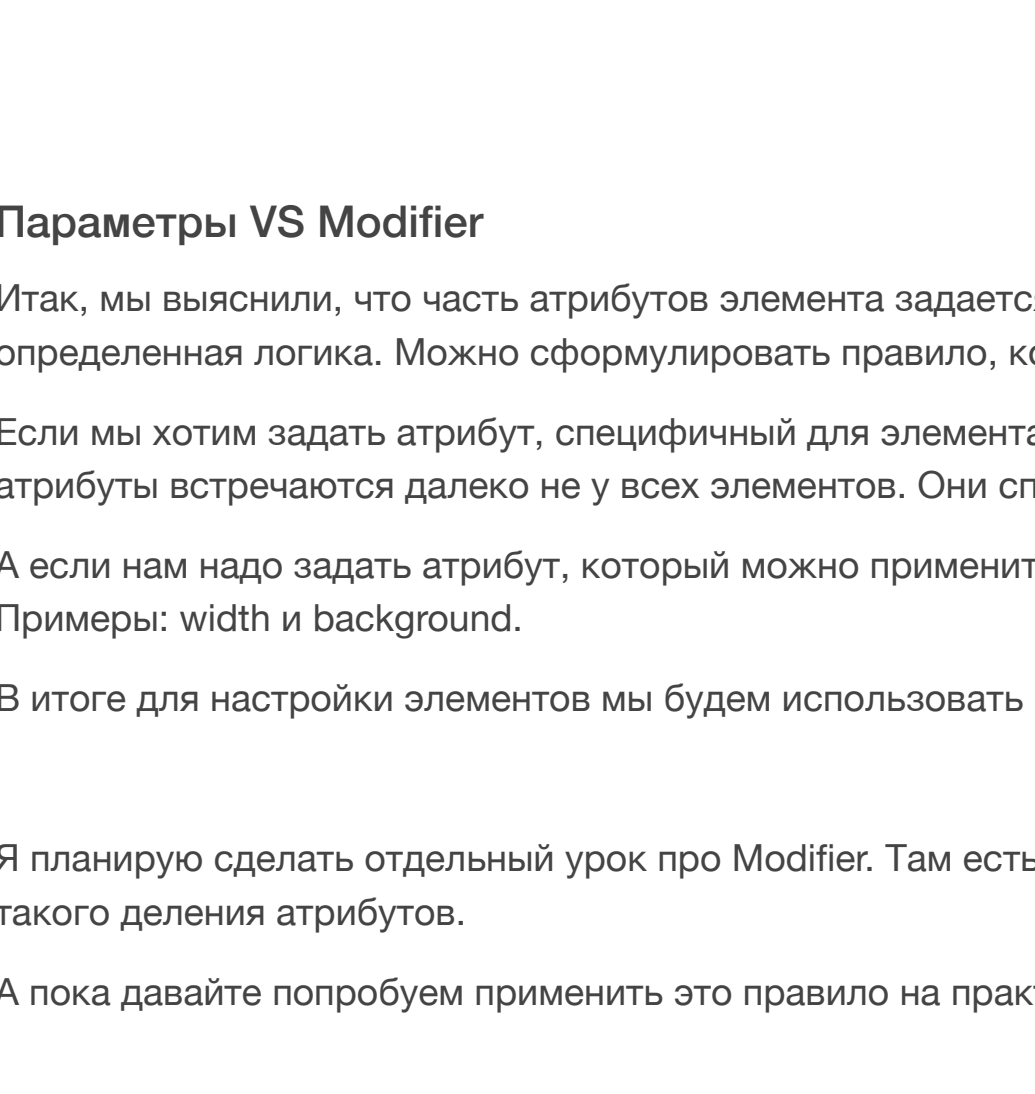
Кроме этого зададим ширину элемента Text в 150 dp

```
import androidx.compose.foundation.layout.width
import androidx.compose.ui.unit.dp

@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 32.sp,
        color = Color.White,
        modifier = Modifier
            .background(color = Color.Black)
            .width(150.dp)
    )
}
```

Обратите внимание, что принцип работы Modifier похож на билдер (хотя и не является таковым). Мы вызываем его методы для передачи значений, и в итоге получим экземпляр Modifier, который содержит необходимые нам настройки элемента.

Результат:



Т.к. ширина элемента стала меньше, текст не влез и перешел на новую строку.

Параметры VS Modifier

Итак, мы выяснили, что часть атрибутов элемента задается параметрами функции, а часть - с помощью Modifier. В этом есть определенная логика. Можно сформулировать правило, которое позволит понять, где искать нужный нам атрибут.

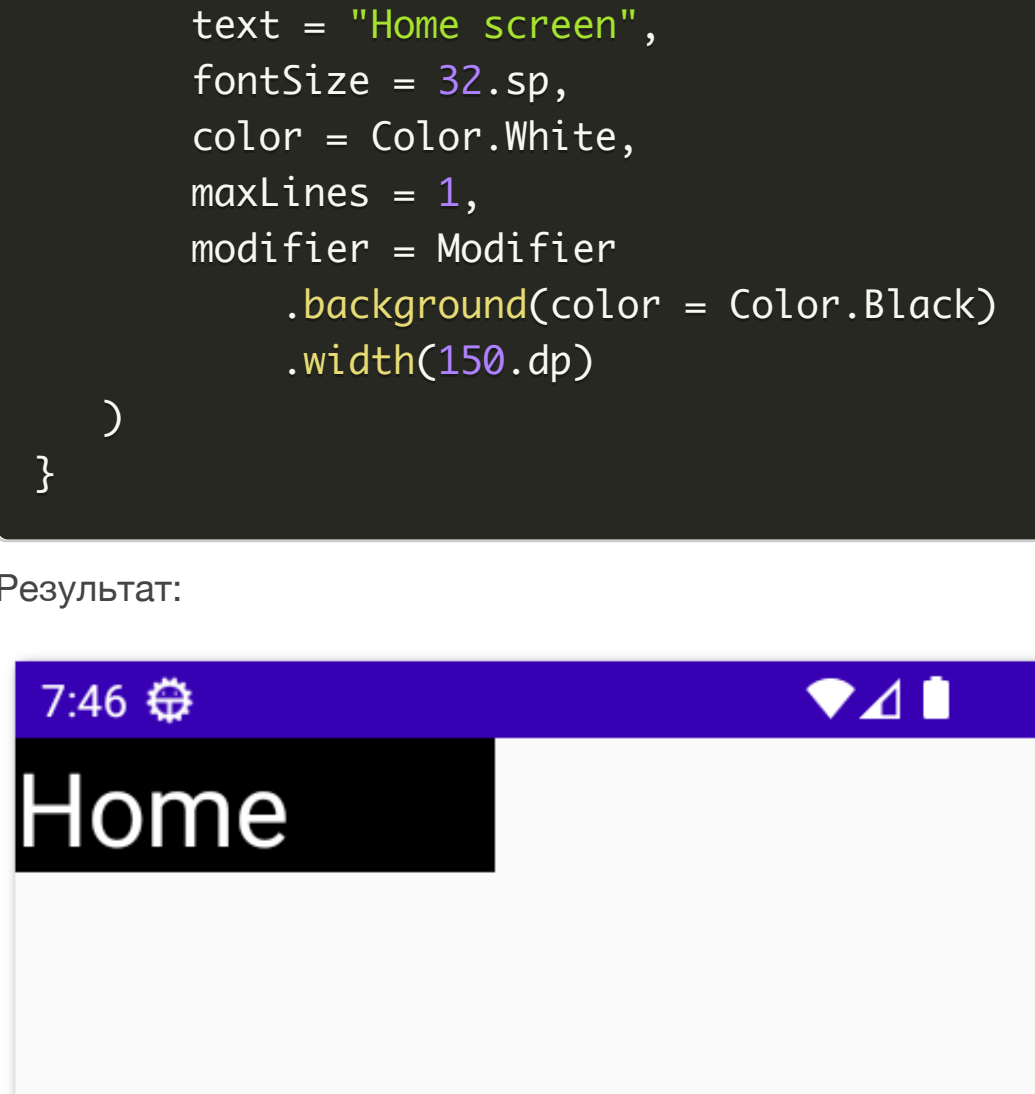
Если мы хотим задать атрибут, специфичный для элемента, то он будет в параметрах функции. Примеры: text и fontSize. Такие атрибуты встречаются далеко не у всех элементов. Они специфичны для элемента Text.

А если нам надо задать атрибут, который можно применить практически к любому элементу, то они будут в Modifier. Примеры: width и background.

В итоге для настройки элементов мы будем использовать и параметры функции и Modifier.

maxLines

Когда мы явно задали ширину элемента, у нас выполнился перенос текста на новую строку.

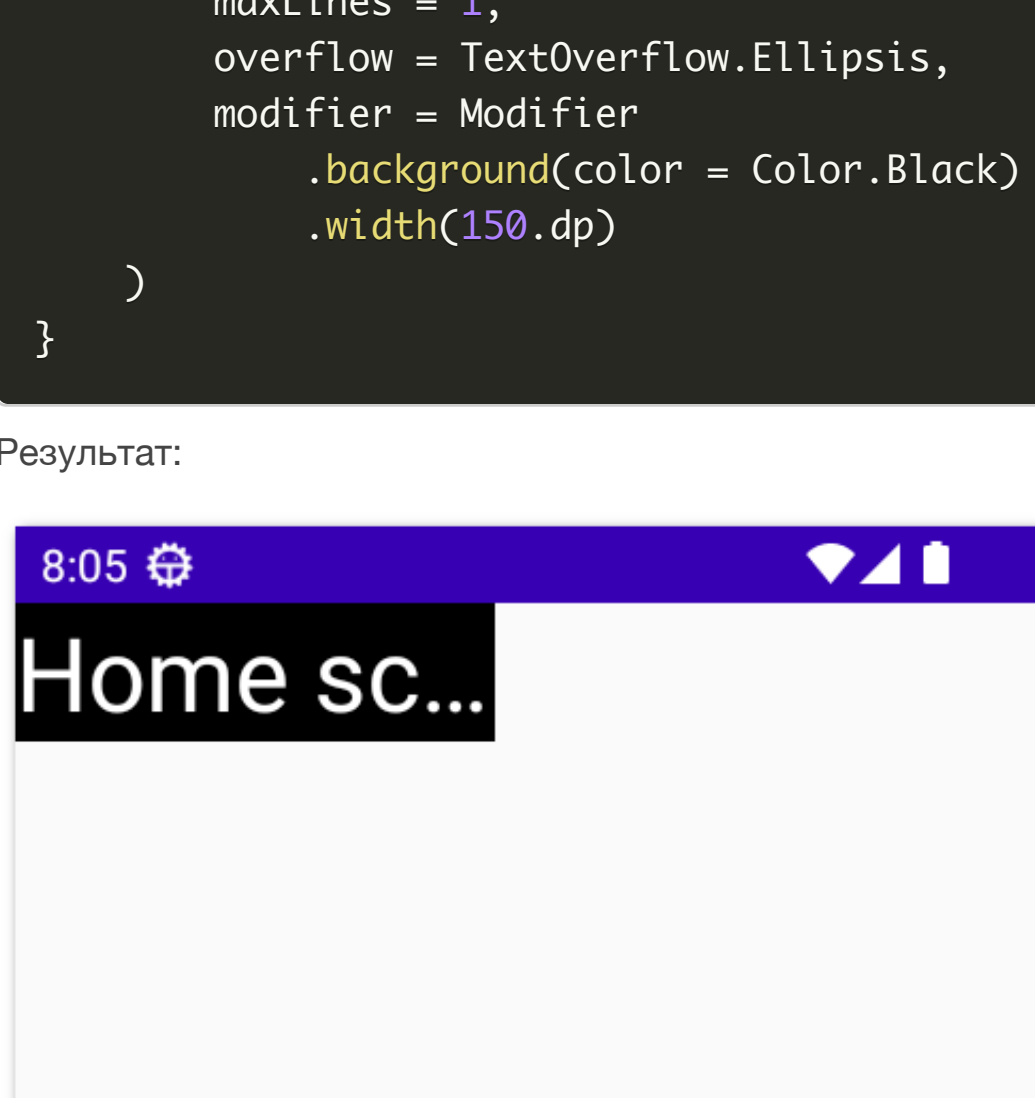


Если мы хотим убрать перенос и принудительно сделать текст в одну строку, то нужно искать знакомый нам по TextView атрибут maxLines. Где его искать? В параметрах функции Text() или в Modifier? Смотрим правило.

Атрибут maxLines не получится задать любому элементу. Он является специфичным для Text. А значит должен быть в параметрах функции Text. И он там есть. Передадим ему значение 1, чтобы текст был в одну строку:

```
@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 32.sp,
        color = Color.White,
        maxLines = 1,
        modifier = Modifier
            .background(color = Color.Black)
            .width(150.dp)
    )
}
```

Результат:



overflow

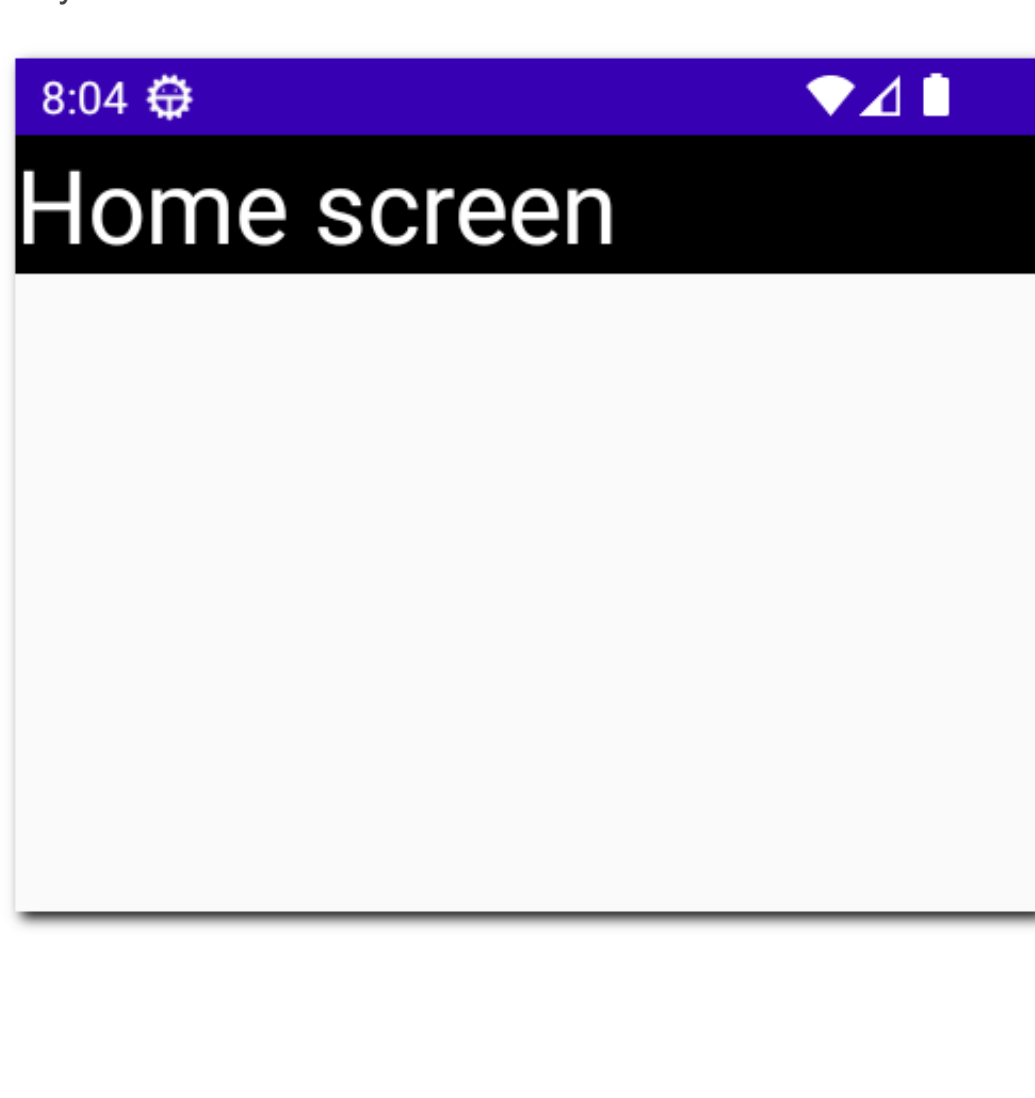
Но теперь слова обрезаются. Хорошо бы сделать так, чтобы он показал часть слова, которое не влезло. Нужен атрибут, который это регулирует. Где его искать?

Это тоже явно не общий для всех элементов атрибут. Он относится к тексту. Поэтому ищем его в параметрах функции Text. Там есть параметр overflow:

```
import androidx.compose.ui.text.style.TextOverflow

@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 32.sp,
        color = Color.White,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
        modifier = Modifier
            .background(color = Color.Black)
            .width(150.dp)
    )
}
```

Результат:



width

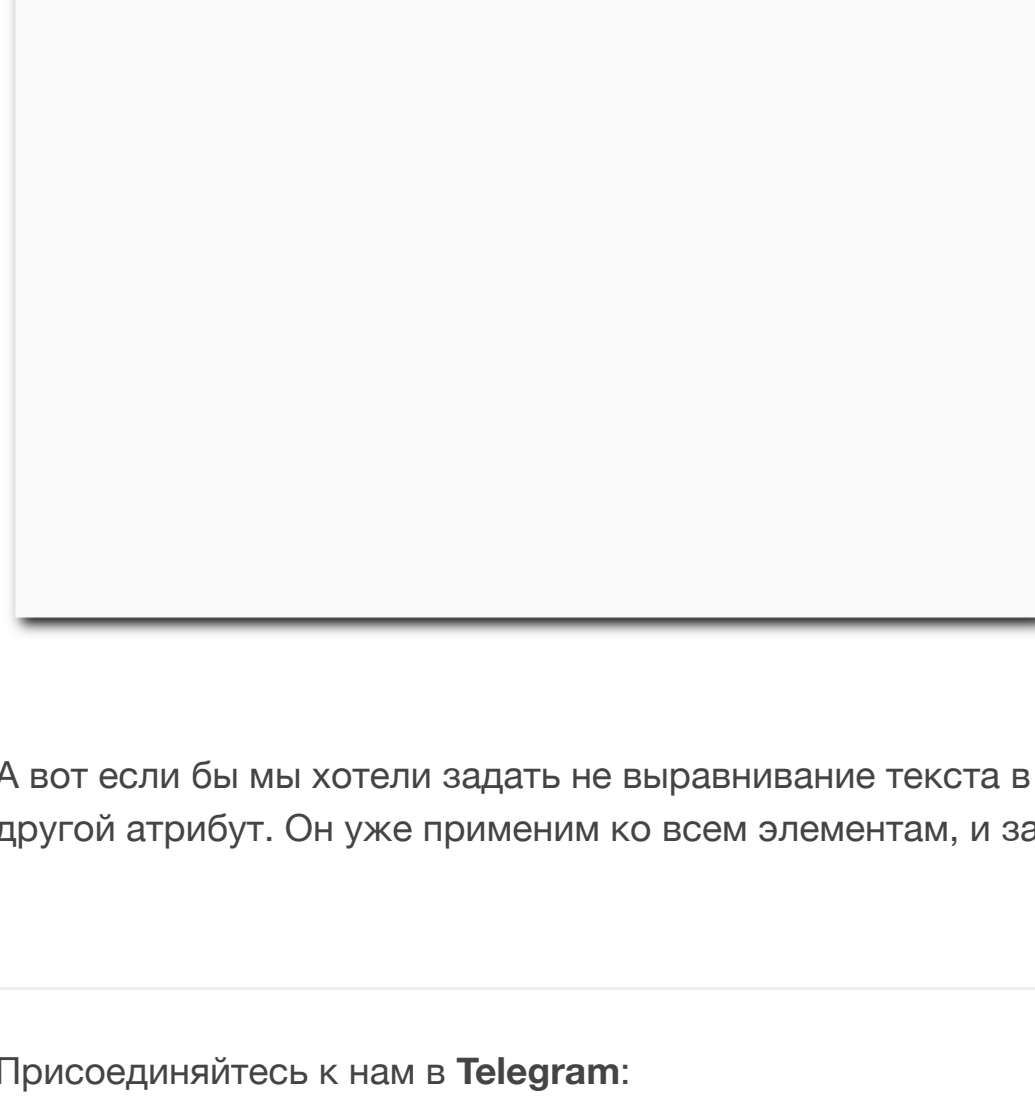
Давайте снова поменяем ширину элемента. Сделаем ее на весь экран.

Ширина - это атрибут, который можно применить к любому элементу, значит она задается в Modifier. Мы это уже делали в начале урока. Но теперь вместо width используем fillMaxWidth.

```
import androidx.compose.foundation.layout.fillMaxWidth

@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 32.sp,
        color = Color.White,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
        modifier = Modifier
            .background(color = Color.Black)
            .fillMaxWidth()
    )
}
```

Результат:



textAlign

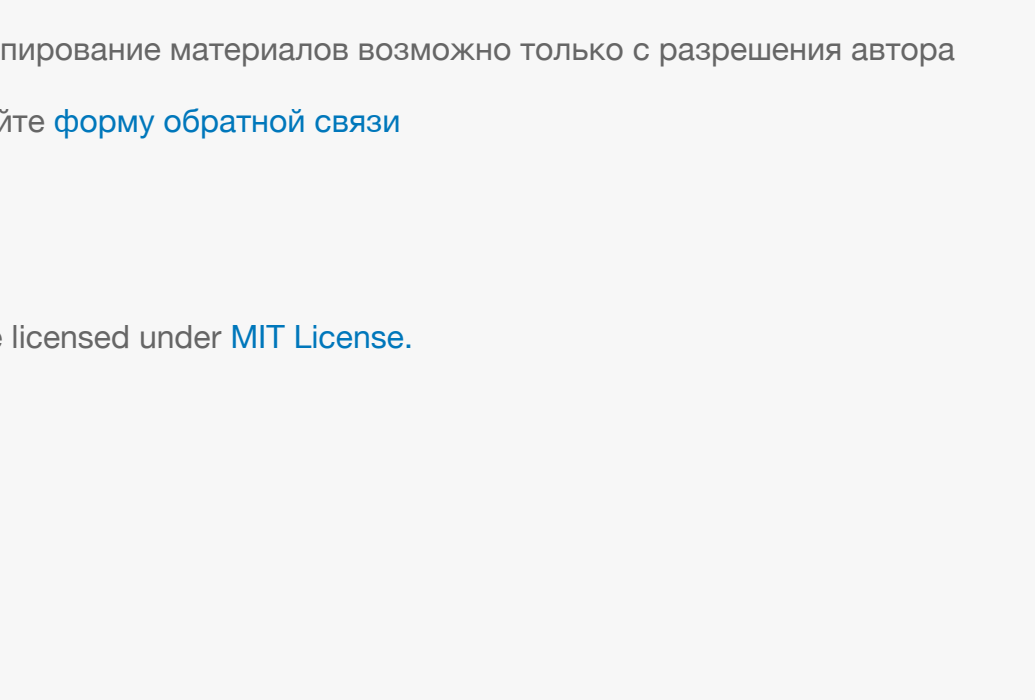
Ну и напоследок сделаем выравнивание, чтобы текст отображался в центре элемента. Атрибут выравнивания текста в элементе не является общим для всех элементов. Он специфичен для Text. Поэтому ищем в параметрах функции.

Находим параметр textAlign:

```
import androidx.compose.ui.text.style.TextAlign

@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 32.sp,
        color = Color.White,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
        textAlign = TextAlign.Center,
        modifier = Modifier
            .background(color = Color.Black)
            .fillMaxWidth()
    )
}
```

Результат:



А вот если бы мы хотели задать не выравнивание текста в элементе, а выравнивание элемента на экране, то это уже совсем другой атрибут. Он уже применим ко всем элементам, и задается в Modifier. Мы его рассмотрим в уроке про Layout.

Присоединяйтесь к нам в **Telegram**:

- в канале [StartAndroid](#) публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.
- в чатах решаем возникающие вопросы и проблемы по различным темам: [Android](#), [Compose](#), [Kotlin](#), [RxJava](#), [Dagger](#), [Тестирование](#), [Performance](#)
- ну и если просто хочется поговорить с коллегами по разработке, то есть чат [Флудилля](#)

Назад Вперед

Добавить комментарий

Тема

👍 👎 😄 😊 😋 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😼 😽 😾 😿 😽 😼 😽 😾 😿

Отправить